Lab 1 · Modelado OLTP → OLAP

PayTrack Colombia: esquema estrella, carga ELT y comparación de rendimiento

SI7006 Almacenamiento y Procesamiento de Grandes Datos · Maestría en Ingeniería de Datos, EAFIT

Integrantes: Samuel Gutiérrez Jaramillo · Alejandro Rendón Correa

---

1. Diagnóstico del esquema OLTP (Parte A)

Se ejecutaron las tres consultas analíticas de `oltp_problems.sql` con `EXPLAIN ANALYZE` sobre 50.000 y 500.000 transacciones. Resumen de los planes:

Consulta	Plan dominante (50K)	Plan dominante (500K)	t_{50K}	t_{500K}
Q1	Bitmap Index Scan (`created_at`) → Hash Join con `merchants` → HashAggregate	Parallel Seq Scan → Hash Join (2 workers)	34,3 ms	46,6 ms
Q2	Seq Scan de `transactions` (sin filtro) → Hash Join con `customers`	Parallel Seq Scan (tabla completa) → Hash Join	13,9 ms	57,7 ms
Q3	Bitmap Index Scan (7 días) → Sort sobre `EXTRACT(...)` → GroupAggregate	Ídem; Sort de 3,3 MB sobre 41.424 filas	4,6 ms	51,8 ms

Q1 (monto por categoría). El filtro de fecha sí aprovecha `idx_txn_created_at`, pero `category` vive en `merchants`: la 3FN obliga a recomponer mediante un *join* la información que el diseño deliberadamente separó. El filtro `status='approved'` no tiene índice propio y se evalúa fila por fila. A 182M de filas (500K/día × 365), el rango de 30 días cubriría ∼60M de filas unidas y agregadas en cada consulta; el tiempo crece con las filas procesadas, no con las 7 filas devueltas.

Q2 (tasa de fraude por segmento). Es el peor caso: no hay filtro de fecha, por lo que se escanea la tabla completa para devolver 3 filas (50.000 filas procesadas → 3 devueltas a 50K; 500.000 → 3 a 500K), y se une todo el histórico con `customers` solo para leer `segment`. Con 182M de filas el *hash join* excedería la memoria y caería a disco temporal, con tiempos del orden de decenas de segundos.

Q3 (día de la semana y hora). El OLTP solo guarda un timestamp: `EXTRACT(DOW/HOUR)` debe calcularse fila por fila y *ordenarse* antes de agrupar, y ninguna función calculada aprovecha un índice. Con 7 días × 500K = 3,5M de filas, el ordenamiento excedería la memoria y usaría disco. En resumen, el OLTP está diseñado para escribir, no para agregar: cada consulta repaga el mismo trabajo porque no conserva resultados agregados.

2. Diseño del esquema estrella (Parte B)

Proceso y grano. El proceso modelado es el registro de transacciones de pago; el grano de `fact_transactions` es **una fila por cada transacción procesada** (ni resumen diario, ni evento de fraude, ni movimiento bancario).

Dimensiones. Cinco dimensiones: `dim_date`, `dim_time` (calendario precalculado: día de la semana, fin de semana, período del día, sin `EXTRACT` en cada consulta), `dim_customer`, `dim_merchant` y `dim_payment_method`. Del OLTP *no* pasan al DW los datos personales del cliente (`name`, `email`, PII) ni el `method_id` serial (llave técnica inestable). Como atributo conformado se agrega `method_type` (tarjeta / billetera digital / transferencia bancaria), inexistente en el OLTP.

Medidas. `amount_cop`, `is_fraud` e `is_approved` son **aditivas** (sumables en cualquier dimensión); `fraud_risk_score` es **no aditiva** (promediable, no sumable) y puede ser nulo: no toda transacción tiene evento de fraude.

Surrogate keys. Se usan llaves sustitutas numéricas en lugar de las llaves del OLTP porque: (1) permiten el versionamiento SCD2, donde un cliente ocupa varias filas; (2) desacoplan el DW de los sistemas operativos; (3) reducen el tamaño en disco y aceleran los *join* (INTEGER vs. VARCHAR).

SCD Tipo 2 en cliente. El cliente puede cambiar de ciudad o de segmento. Si se sobrescribe el valor, todas las transacciones pasadas "cambiarían de segmento" retroactivamente y la tasa de fraude histórica quedaría distorsionada. SCD2 conserva cada versión con `valid_from`/`valid_to` y marca la vigente con `is_current`; por eso la PK no puede ser `customer_id` (sería no única entre versiones) sino la surrogate key `customer_key`. En la carga inicial: `valid_from` = creación del cliente, `valid_to` = `'infinity'`, `is_current` = `TRUE`. El comercio es SCD1 (atributos estables); el método de pago conserva `method_name` como llave natural.

Índices de la hechos. `date_key` (Q1/Q3), `merchant_key` (Q1), `customer_key` (Q2) y un índice parcial `merchant_key WHERE is_fraud` para análisis de solo fraude; su efecto real se midió con `EXPLAIN ANALYZE` (Sección 4).

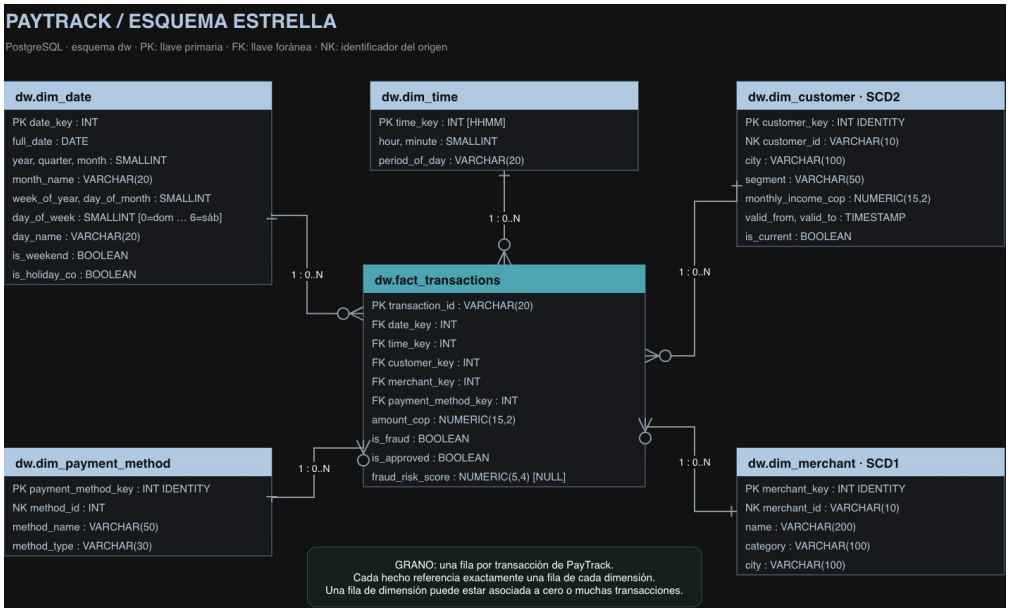


Figure 1: Esquema estrella PayTrack: 5 dimensiones, tabla de hechos y SCD2 de cliente.

3. Carga ELT y validación (Parte C)

El proceso ELT (4 `INSERT ... SELECT` de `oltp.*` a `dw.*`) traduce cada llave natural a su surrogate key; para el cliente une la transacción con la versión vigente en su fecha (`created_at` entre `valid_from` y `valid_to`), deriva `date_key` (AAAAMMDD), `time_key` (HHMM) e `is_approved` = (`status='approved'`), y trae `risk_score` mediante **LEFT JOIN** con `fraud_events` (solo ∼0,4% de las transacciones tiene evento; un INNER JOIN habría descartado el resto). `verify()` valida conteos, montos, fraudes e integridad referencial.

```
PS C:\Users\Samuel\Documents\UNIVERSIDAD EAFIT\LINEA_DE_ENFASIS\Almacenamiento y Procesamiento de Grandes Datos\big-data-labs-2026-2> uv run python labs/01-
modelado/load_star.py
✓ Esquema estrella creado
✓ dim_date: 91 filas
✓ dim_time: 1440 filas
✓ dim_customer: 500 filas
✓ dim_merchant: 8 filas
✓ dim_payment_method: 5 filas
✓ fact_transactions: 500000 filas

Validando la carga...
  ✓ dim_customer tiene un registro vigente por cliente
  ✓ dim_merchant tiene todos los comercios
  ✓ dim_payment_method tiene todos los métodos
  ✓ fact_transactions tiene una fila por transacción
  ✓ el monto total cuadra
  ✓ el número de fraudes cuadra
  ✓ el número de transacciones aprobadas cuadra
  ✓ toda transacción tiene una fecha y hora válidas en las dimensiones
  ✓ toda transacción tiene cliente, comercio y método en las dimensiones

¡Esquema estrella cargado y validado!
```

Figure 2: `load_star.py`: 9 validaciones en verde (500K filas).



Figure 3: `check_queries.py`: 3/3.

4. Consultas OLAP, comparación de rendimiento e índices (Parte D)

Validación. Las tres consultas se reescribieron usando únicamente `dw.*`, con las mismas columnas que la versión OLTP. El validador confirmó resultados idénticos: Q1 7 filas, Q2 3 filas, Q3 168 filas, verificado tanto a 50.000 como a 500.000 transacciones. En Q2 se agrupa por el segmento *actual* del cliente: se lo identifica desde la versión asociada al hecho y se busca su versión vigente (`is_current`), replicando el OLTP sin excluir transacciones históricas.

Comparación de rendimiento. Cinco repeticiones por modelo con estadísticas actualizadas y orden alternado; medianas:

	50.000 transacciones		500.000 transacciones	
Consulta	**OLTP**	**OLAP**	**OLTP**	**OLAP**
Q1 (monto por categoría)	10,1 ms	14,6 ms	46,6 ms	54,3 ms
Q2 (tasa de fraude)	19,5 ms	18,2 ms	57,7 ms	60,5 ms
Q3 (día/hora)	5,9 ms	9,6 ms	51,8 ms	75,1 ms

Extracto de los planes de Q1 con `EXPLAIN ANALYZE` a 500K (misma consulta en ambos modelos; los planes completos están en `labs/01-modelado/evidencias/`):

```
-- OLTP
Finalize GroupAggregate (actual time=42.4..46.2
  rows=7)  Group Key: m.category
  ->  Gather Merge (2 workers)
      ->  Hash Join (actual time=0.09..30.7
           rows=56358 loops=3)
          ->  Parallel Seq Scan on transactions t
               Filter: (status='approved') AND
                 (created_at >= CURRENT_DATE-30)
               Rows Removed by Filter: 110309
Execution Time: 46.589 ms
```

```
-- OLAP
Finalize GroupAggregate (actual time=47.9..54.0
  rows=7)  Group Key: m.category
  ->  Gather Merge (2 workers)
      ->  Hash Join (f.merchant_key =
           m.merchant_key) rows=56358
          ->  Hash Join (f.date_key = d.date_key)
               ->  Parallel Seq Scan on
                    fact_transactions f
                    Filter: is_approved
Execution Time: 54.303 ms
```

En Q1 y Q3 a 50K, el OLTP usó el índice de fecha (17.006 y 4.284 filas del período); el OLAP recorrió los 50.000 hechos y filtró mediante *join*: el planificador prefirió un *sequential scan* sobre `fact_transactions` en vez de `idx_fact_date`, decisión razonable en tablas pequeñas. A 500K, Postgres activó dos trabajadores paralelos y ambos modelos escalaron de manera sublineal ($10\times$ datos $\rightarrow \sim 5\times$ tiempo), manteniéndose las diferencias en el mismo rango de magnitud.

Índices. Retirarlos individualmente produjo variaciones inferiores al 1,6%: ninguno apareció en los planes iniciales, porque con tablas de este tamaño recorrer la tabla completa resulta más barato que el acceso por índice. El índice parcial de fraude no cubre por sí solo Q2, pues la tasa también requiere contar las transacciones no fraudulentas. Análogamente, `is_approved` conserva el 99,87% de las filas (Q2 necesita todas), lo que explica la preferencia por el *sequential scan*.

Conclusión. El esquema estrella produce resultados correctos, validados uno a uno contra el OLTP a ambas escalas. En este experimento el acceso por índice de fecha favoreció al OLTP en Q1 y Q3, y Q2 fue ligeramente más rápida en OLAP a 50K (equivalente a 500K); como advierte el propio README, con estas escalas las diferencias son pequeñas. El valor del modelo dimensional no se limita al tiempo de respuesta: garantiza acceso directo a los atributos de agrupación (categoría, segmento, día, hora) sin recalcularlos ni unir tablas normalizadas, y conforme el volumen crece hacia las 500K transacciones diarias del caso de uso, las agregaciones sobre dimensiones precalculadas escalan de manera más predecible que los *join* repetidos sobre un esquema OLTP.